

ON-PREMISES ENTERPRISE AI PLATFORM

Hosting GLM-5.2 Inference: Feasibility & Recommended Architecture

Why the CPU-only blade estate powers the AI platform — and a dedicated GPU server runs the model

Director Decision Briefing Prepared by Enterprise Architecture · July 2026

The Decision in One View

GO

Build the Kubernetes AI platform on the 6 blades

Ample CPU/RAM for a hardened, highly-available microservices platform hosting all AI solutions.

GO

Run embeddings, rerankers & small/mid models on blades

CPU-friendly workloads that add real value today and preserve scarce GPU capacity.

NO-GO

Interactive full GLM-5.2 on the CPU-only blades

744B-parameter model; ~460-480 GB just for weights; memory bandwidth caps speed to unusable levels.

PLAN

Add a separate GPU server for GLM-5.2 inference

Blades call it over a secure, swappable API. Preserves the whole platform investment.

Why CPU-Only Cannot Host GLM-5.2

Kubernetes does not merge RAM across servers — a model must fit inside ONE machine's memory and bandwidth.

744B

GLM-5.2 parameters (MoE)

**~460–480
GB**

Weights alone at 4-bit

~341 GB*

RAM per blade (assumed)

Single-digit

Tokens/sec best case on CPU

The math: the model won't fit one blade even quantized, and decode speed is limited by memory bandwidth (not core count). Spreading it across blades over Ethernet is slow and not fault-tolerant. * *Per-blade RAM is an assumption pending hardware discovery.*

What Is Viable on This Estate

Capability	Classification
Kubernetes microservices platform (blades)	Production viable
Embedding & reranking models (blades, CPU)	Production viable
Small / mid models, replicated (blades, CPU)	Viable — validate latency
Full GLM-5.2, GPU server	Recommended path
Full GLM-5.2, interactive on CPU blades	Rejected
Full GLM-5.2 spread across blades (Ethernet)	Benchmark-only, not HA

Recommended Architecture: Two Planes

The blades run everything around the model; a separate GPU server runs the model. They connect over a secure API.

PLANE 1 — Blades (CPU) • Kubernetes AI Platform

- API gateway • authN/Z • quotas • audit
- Model router (OpenAI-compatible)
- RAG / orchestration / agents
- Vector DB • cache • ingestion
- Embeddings & rerankers (CPU)
- Registry • secrets • GitOps • observability



mTLS API

PLANE 2 — GPU Server • Inference

- GLM-5.2 on GPUs (vLLM/SGLang)
- Tensor-parallel over NVLink
- OpenAI-compatible HTTPS endpoint
- No content logging
- Independently scalable & swappable

All data & prompts stay on-premises • GPU server is a swappable backend behind the model router

GPU Server Sizing — Illustrative

Interactive worksheet provided; confirm against the pinned engine before procurement.

INT4 / Q4

~476 GB weights

6–8× 80 GB GPUs

FP8

~740–800 GB weights

8× 80 GB or 1–2 nodes

Throughput

~250+ tok/s

aggregate, GPU-served

Add KV-cache VRAM for concurrency and context. A second GPU server removes the single point of failure and roughly doubles throughput.

Delivery Roadmap

1

Discover & size

Blade hardware discovery; GPU procurement sizing; agree SLOs.

2

Build platform

RKE2 K8s, security, observability, GitOps on the 6 blades.

3

Stand up GPU plane

GPU server + vLLM serving GLM-5.2 behind a secured API.

4

Integrate & test

Wire microservices to GPU; load, failover & DR testing.

5

Onboard production

Roll out AI solutions by tier with quotas and monitoring.

Key Risks & How We Manage Them



Single GPU server = outage risk for the big model

Add a 2nd GPU backend; degrade to a smaller model or queue on failure.



GPU sizing uncertainty (VRAM + KV-cache)

Benchmark before go-live; worksheet models precision & concurrency.



Data confidentiality

On-prem only; mTLS between planes; prompts/responses never logged by default.



Supply-chain integrity (weights & images)

Signed, checksummed artifacts; verified before load; air-gap import.



Model support may lag release

Verify GLM-5.2 on pinned engine; fall back to supported variant.

Recommendation & Decisions Requested

Proceed with the two-plane architecture: build the platform now, add GPU inference in parallel.

- 1 **Approve** the Kubernetes AI platform build on the existing 6 blades (no new hardware).
- 2 **Fund** one GPU inference server (with a 2nd for HA on the roadmap).
- 3 **Authorize** Phase 1 hardware discovery to replace assumptions with measured facts.
- 4 **Confirm** priority AI use cases, SLOs and data-handling requirements with owners.

Outcome: production-grade, secure, on-prem AI — honest about today's limits, ready to scale with GPU.